

```
# Step 1 : Install Required Libraries
# pip install tensorflow keras numpy matplotlib scikit-learn
```

```
# Step 2: Import Libraries
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Conv2D, MaxPooling2D
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from keras.datasets import mnist
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report
```

```
#Step 3: Load and Preprocess the MNIST Dataset
# Load the MNIST dataset
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

# Normalize and reshape the images
train_images = train_images.reshape((60000, 28, 28, 1)).astype('float32') / 255
test_images = test_images.reshape((10000, 28, 28, 1)).astype('float32') / 255

# One-hot encode the labels
train_labels = keras.utils.to_categorical(train_labels, 10)
test_labels = keras.utils.to_categorical(test_labels, 10)
```

```
# Step 4: Set Up Data Augmentation
datagen = ImageDataGenerator(
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

# Fit the data generator to the training images
datagen.fit(train_images)
```

```
# Step 5: Build the Neural Network Model
model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(Flatten())
model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))

# Display the model's architecture
model.summary()
```

[Show hidden output](#)

```
# Step 6: Compile and Train the Model
# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train the model with data augmentation
history = model.fit(datagen.flow(train_images, train_labels, batch_size=32),
                    steps_per_epoch=int(len(train_images) / 32),
                    epochs=3,
                    validation_data=(test_images, test_labels))
```

[Show hidden output](#)

```
#Step 7: Plot Training History
# Plot training & validation accuracy values
plt.figure(figsize=(12, 4))
# plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy',loc='left')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.text(1.0,1.02,'(2311cs010480-N.Rishik)',transform=plt.gca().transAxes,ha='right')
plt.legend(['Train', 'Validation'], loc='upper left')
```

[Show hidden output](#)

```
# Plot training & validation loss values
# plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss',loc='left')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.text(1.0,1.02,'(2311cs010480-N.Rishik)',transform=plt.gca().transAxes,ha='right')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```

[Show hidden output](#)

```
#Step 8: Make Predictions and Generate a Classification Report
# Make predictions
predictions = model.predict(test_images)
predicted_classes = np.argmax(predictions, axis=1)
true_classes = np.argmax(test_labels, axis=1)

# Generate a classification report
print(classification_report(true_classes, predicted_classes))
```

[Show hidden output](#)

```
# Step 9: Visualize Some Predictions
# Display a few test images with their predicted and true labels
num_images = 8
plt.figure(figsize=(8, 4))
plt.text(1.0,1.02,'(2311cs010480-N.Rishik)',transform=plt.gca().transAxes,ha='right')
for i in range(num_images):
    plt.subplot(1, num_images, i + 1)
    plt.imshow(test_images[i].reshape(28, 28), cmap='gray')
    plt.title(f"Pred: {predicted_classes[i]}\nTrue: {true_classes[i]}")
    plt.axis('off')
plt.show()
```

[Show hidden output](#)

```
#Step 10: Evaluate the Model on Test Data
# Evaluate the model on the test data
test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f'Test accuracy: {test_acc:.4f}')
```

313/313 ————— 4s 12ms/step - accuracy: 0.9439 - loss: 0.1660
Test accuracy: 0.9535

